

Extending LEXO with Static Analysis: Enriching I/O Pair Generation via Function Metadata

Abdelhak Chellal

June 2026

1 Motivation

During the LEXO reproduction, we observed that some packages were regenerated without implementing their actual logic. The clearest example was `has-proto`, where multiple models produced:

```
module.exports = function () { return true; };
```

This passed all developer tests because the I/O pairs contained only positive cases — the model overfit the examples rather than inferring the real behaviour. The root cause: the input generation prompt receives only raw source code, with no structured information about what the function does, what its parameters mean, or what types they expect.

We extended the pipeline to extract this information statically and inject it into the prompt.

2 Implementation

We added two files to the pipeline:

`extract_signatures.js` parses JavaScript source files using the acorn AST parser and extracts function signatures, parameter names with default values and inferred types, and JSDoc annotations (`@param`, `@returns`, `@throws`, `@example`).

`static_analysis.py` is a unified wrapper that calls `extract_signatures.js` for JS packages and uses Python's built-in `ast` module for Python packages. It exposes `analyze_package()` and `format_metadata_for_prompt()`, which formats the extracted metadata into a structured snippet injected into the `generate_inputs()` prompt. Extracted metadata is also saved to `pipeline/results/<package>/metadata.txt` for inspection. The integration is non-breaking: if analysis fails, the pipeline falls back silently to the original behaviour.

We ran two enrichment experiments.

2.1 Enrichment V1 — Signatures and Documentation

V1 injects function signatures, parameter types, and JSDoc or Python docstrings. For well-documented packages, this provides useful context. For `just-pick`:

```
### 'pick(obj, select)'  
- description: pick(obj, ['a', 'c']); // {a: 3, c: 9}  
               pick(obj, 'a', 'c');    // {a: 3, c: 9}
```

```
- params: 'obj', 'select'
```

For undocumented packages like `has-proto` or `is-number`, the metadata was minimal — only parameter names with no semantic context:

```
### 'exports(num)'  
- params: 'num'
```

This limitation motivated a second iteration.

2.2 Enrichment V2 — Adding Function Body

V2 adds the first 15 lines of the function body to the snippet. For packages whose logic is pre-computed outside the exported function (like `has-proto`, which uses an IIFE), the full module-level context is included instead:

```
### 'hasProto()'  
- body:  
  var test = { __proto__: null, foo: {} };  
  var result = { __proto__: test }.foo === test.foo  
    && !(test instanceof Object);  
  // --- function body ---  
  return result;
```

3 Results

Model	Baseline	V1	V2	V1 Δ	V2 Δ
GPT-5.4 mini	86.8%	99.5%	94.1%	+12.7%	+7.3%
GPT-4o mini	59.7%	62.6%	72.6%	+2.9%	+12.9%
GPT-3.5 Turbo	52.2%	50.4%	47.2%	-1.8%	-5.0%
Mistral Nemo	38.5%	31.7%	23.0%	-6.8%	-15.5%
Claude 3.5 Haiku	83.8%	85.6%	79.2%	+1.8%	-4.6%
Owl Alpha	84.5%	70.2%	66.9%	-14.3%	-17.6%
DeepSeek v4 Flash	80.5%	88.8%	84.4%	+8.3%	+3.9%

Table 1: Average developer test pass rate (ok runs only) across 13 packages. Baseline used `mistral-7b-instruct-v0.1`; enriched experiments used `mistral-nemo` (model became unavailable on OpenRouter).

LEXO Reproduction — Top 4 Models

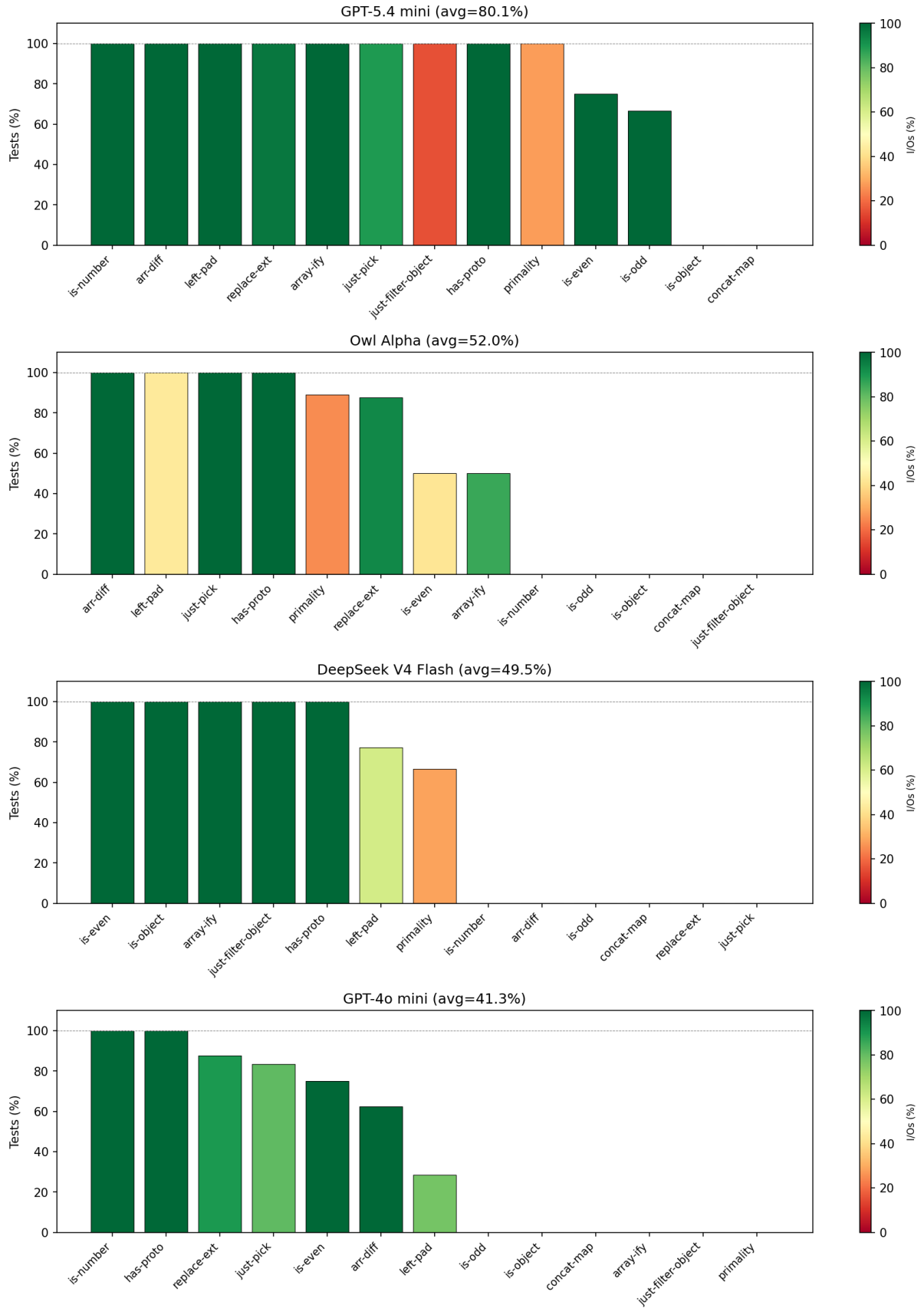


Figure 1: Baseline — top 4 models by average score.

LEXO + Static Analysis — Top 4 Models

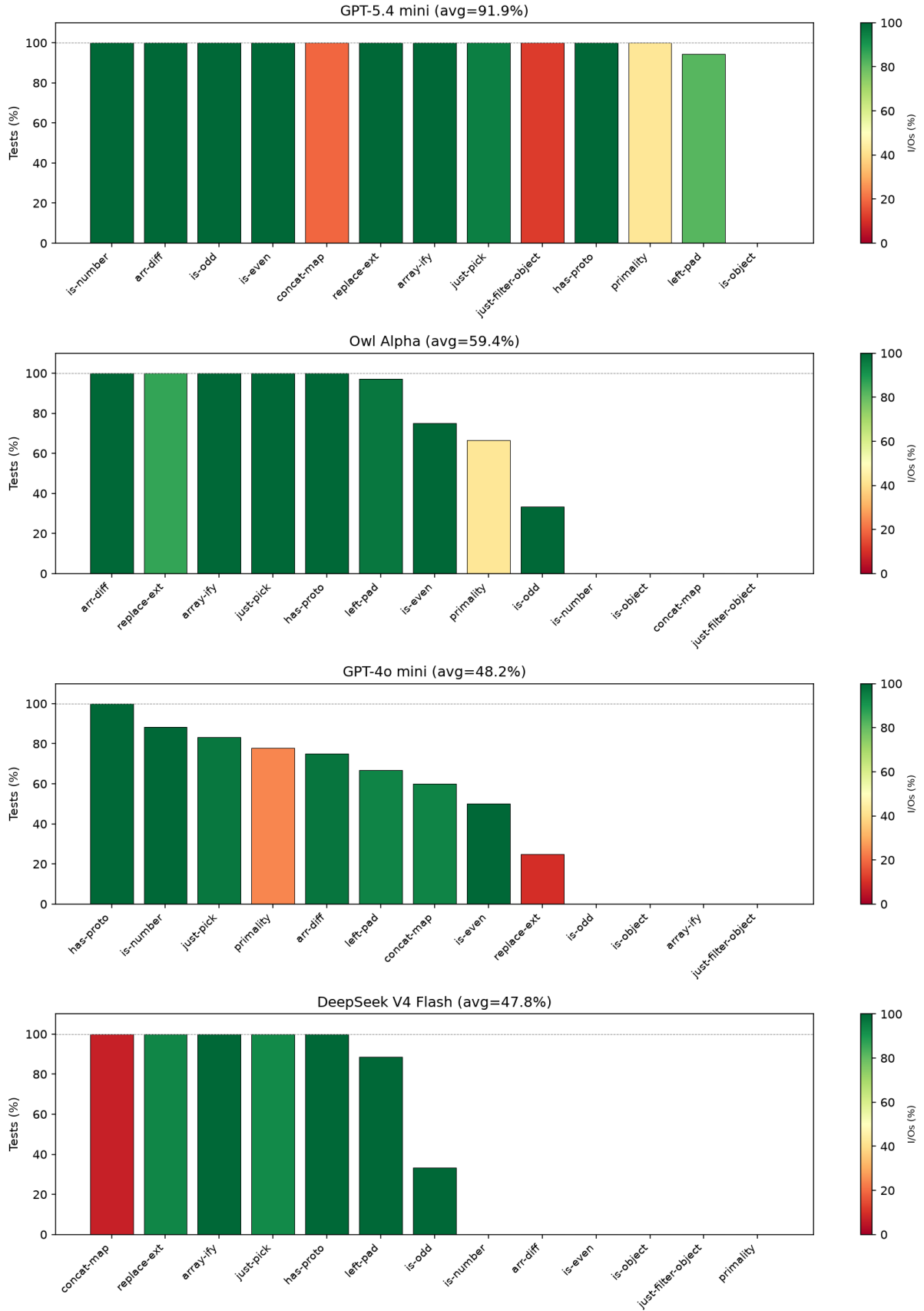


Figure 2: Enrichment V1 (signatures + documentation) — top 4 models.

LEXO + Static Analysis v2 — Top 4 Models

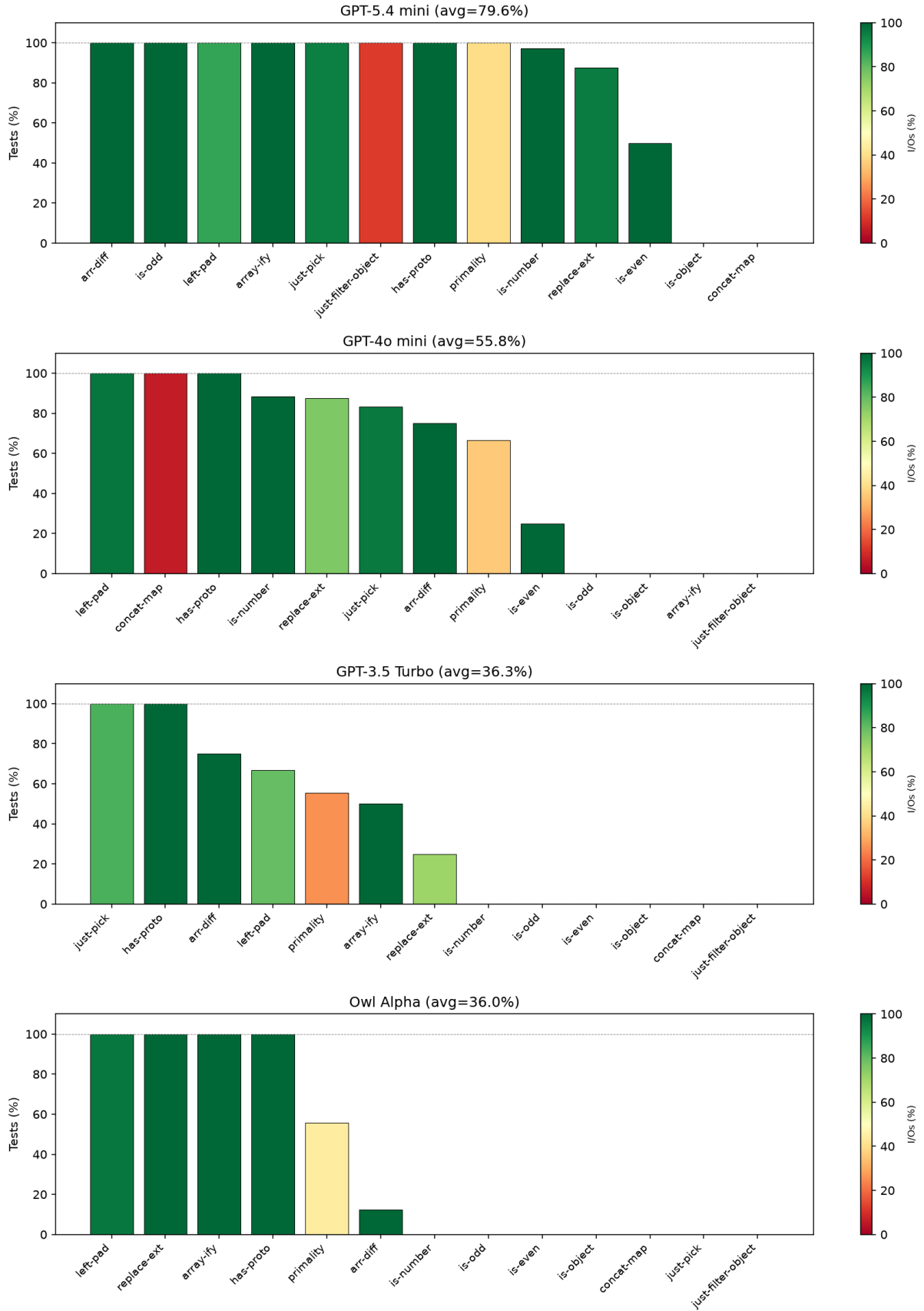


Figure 3: Enrichment V2 (V1 + function body) — top 4 models.

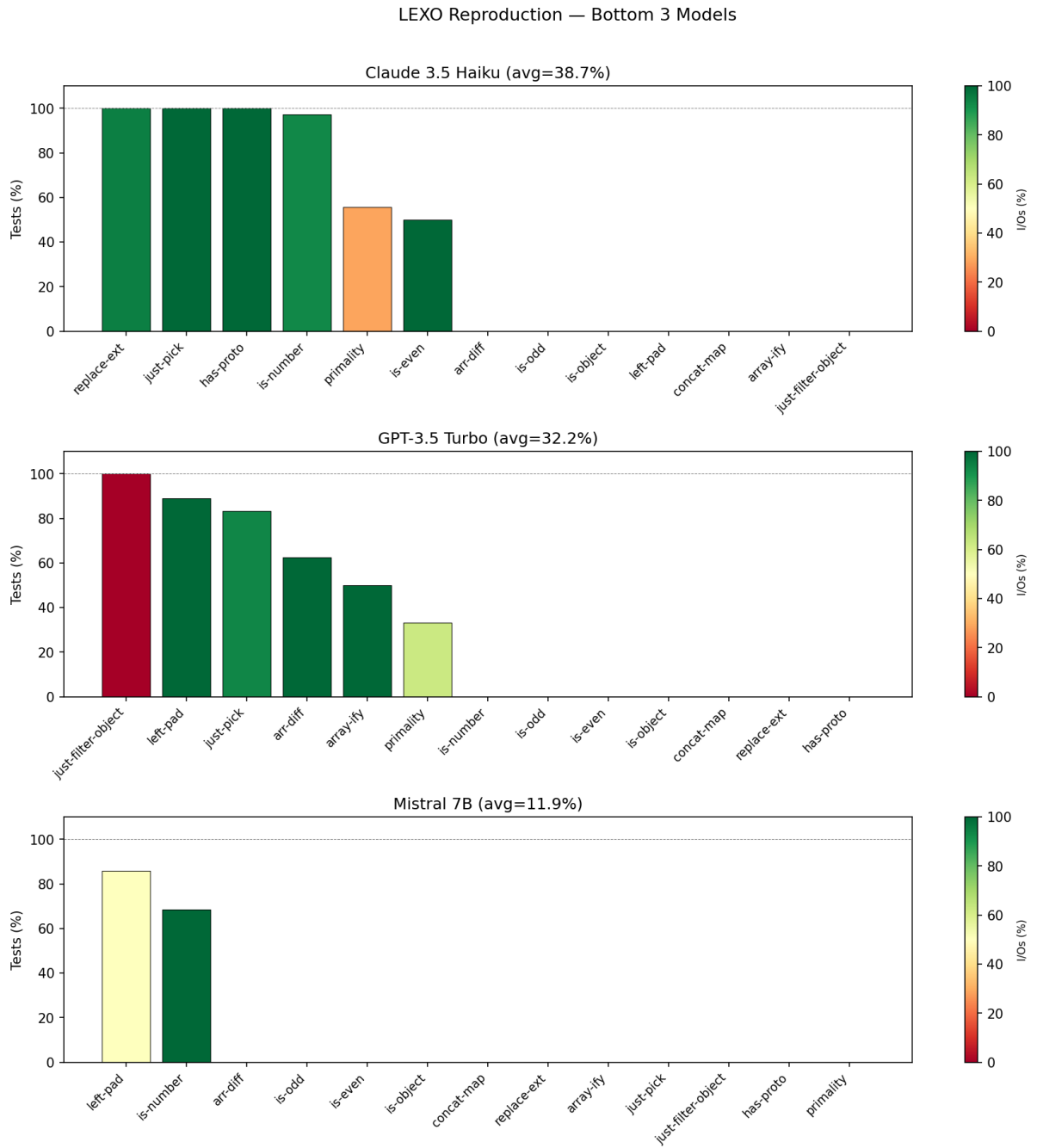


Figure 4: Baseline — bottom 3 models.

LEXO + Static Analysis — Bottom 3 Models

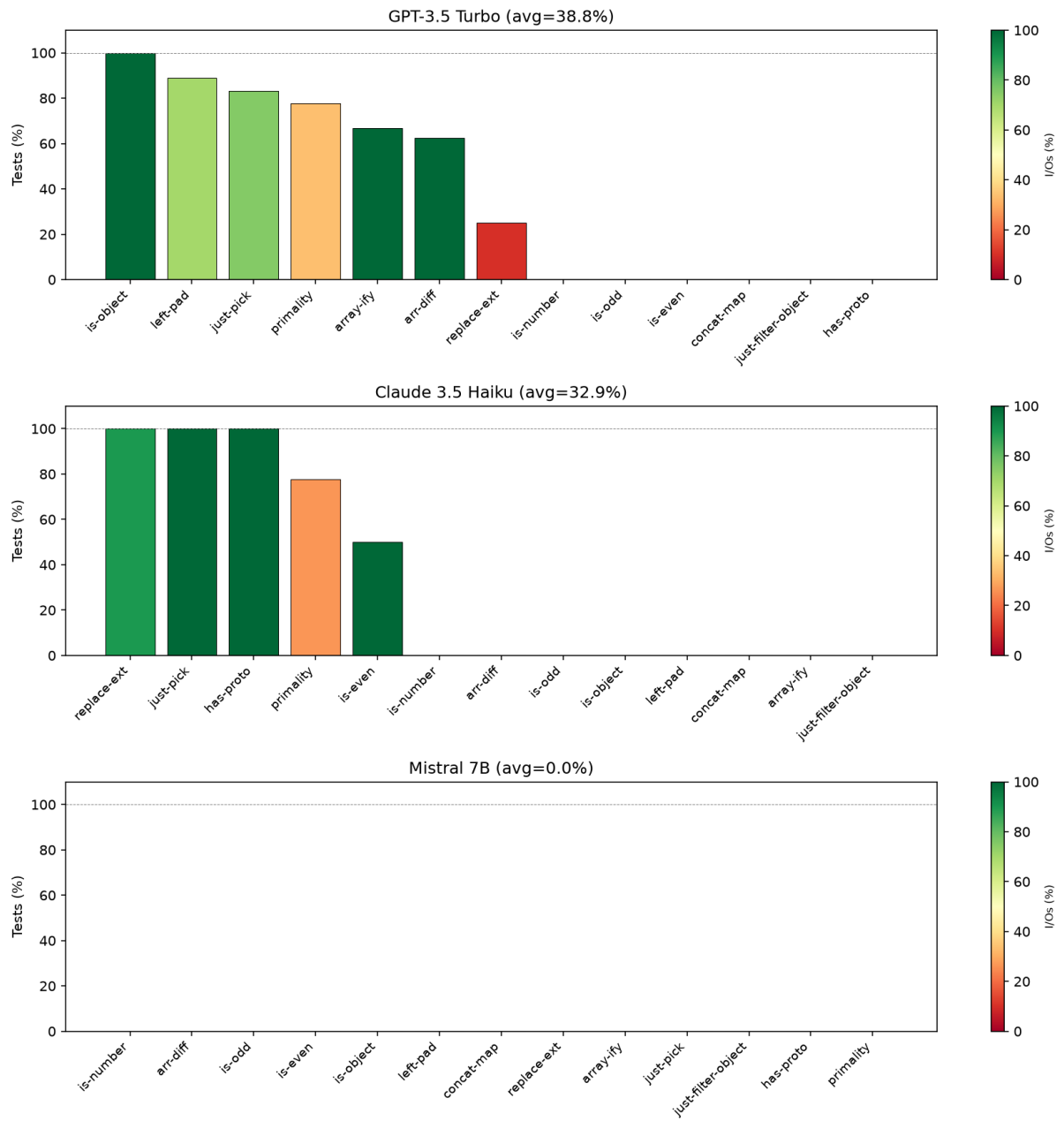


Figure 5: Enrichment V1 — bottom 3 models.

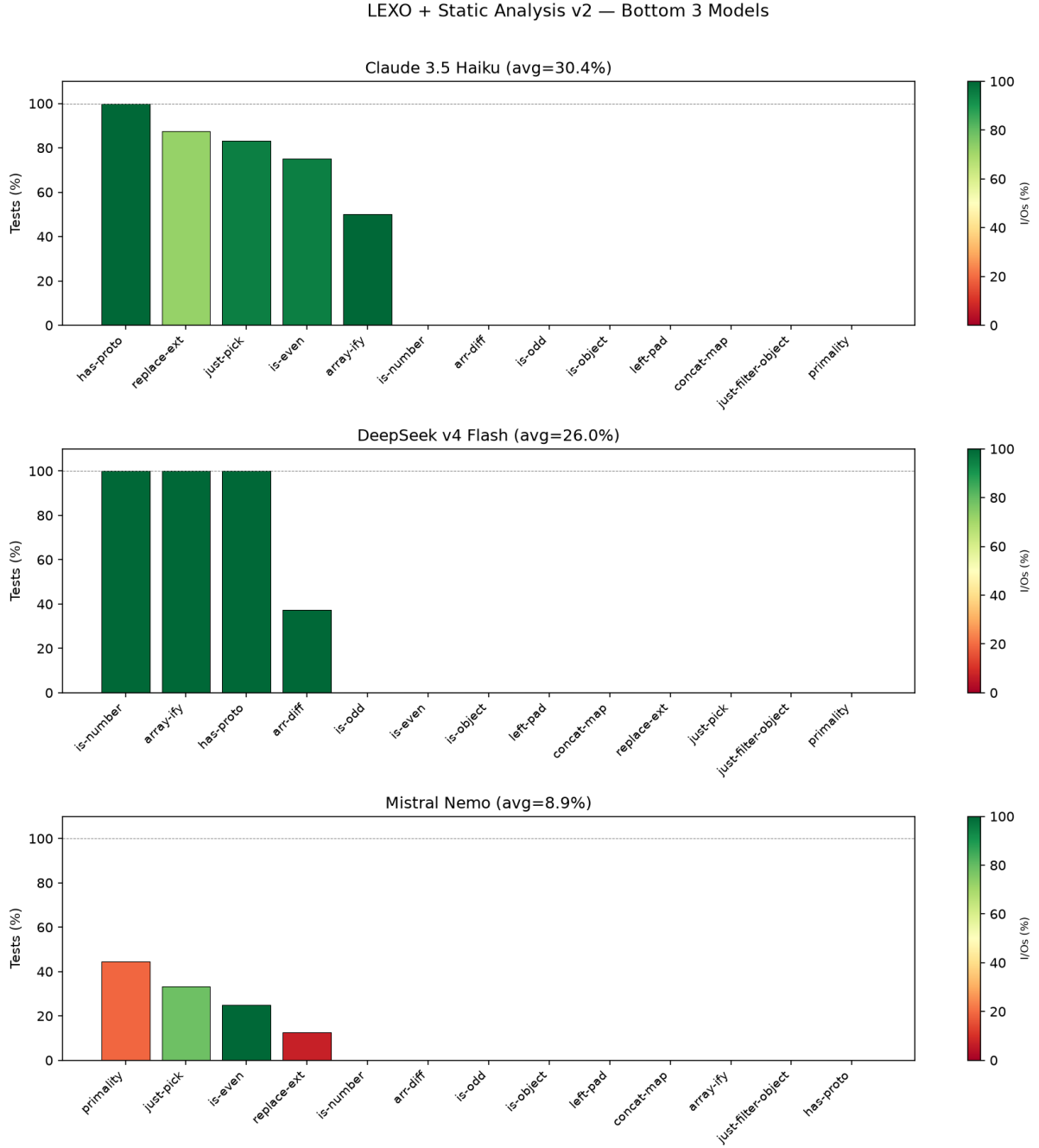


Figure 6: Enrichment V2 — bottom 3 models.

4 Analysis

V1 improves stronger models. GPT-5.4 mini (+12.7%), DeepSeek (+8.3%), and Claude Haiku (+1.8%) all improved. The gains are most visible on packages with rich documentation: `primality` (Python docstrings with type annotations) improved across all capable models — GPT-4o mini from 0% to 77.8%, Claude Haiku from 55.6% to 77.8%, GPT-3.5 Turbo from 33.3% to 77.8%. The metadata gave the LLM explicit context about parameter types, valid ranges, and expected exceptions, resulting in more diverse and semantically grounded test inputs.

V1 hurts weaker models. Owl Alpha (−14.3%) and Mistral Nemo (−6.8%) regressed. The

additional prompt length appears to increase the probability of malformed JSON output in models that already struggle with structured output. The gain in semantic grounding is outweighed by the loss in output reliability.

V2 results are mixed. GPT-4o mini improved further to +12.9% overall, with notable gains on `left-pad` (28.6% → 100%) and `concat-map` (60% → 100%) — cases where seeing the implementation helped. However, error counts increased for Owl Alpha (2 → 6 errors) and DeepSeek (6 → 9 errors), indicating that the body slice increases prompt complexity beyond what these models can reliably handle.

The `has-proto` case. Despite the added module-level context in V2, `has-proto` still scores 100% dev tests across all models in all three runs. This is because the function genuinely always returns `true` in any modern JS engine — the IIFE computes a boolean at module load time that is always `true` in V8. The regenerated `return true` is actually correct. The overfitting problem exists but is invisible to the dev test metric in this specific case.

5 Conclusion

Static analysis enrichment is a lightweight, zero-cost addition to the LEXO pipeline. V1 (signatures + documentation) is the more robust improvement, consistently helping capable models while being transparent about its limitations on undocumented packages. V2 (adding function body) shows selective gains but increases prompt complexity with diminishing returns for most models.

The results confirm that the quality of I/O pair generation is a significant driver of final correctness. Providing structured semantic context improves input diversity, particularly for packages with rich documentation. For undocumented packages, the metadata is too sparse to be impactful — a finding that points toward alternative enrichment strategies such as type inference or documentation generation as preprocessing steps.

Repository: <https://github.com/Abdelhak-Chellal/Lexo-repro>